

Отчёт по модулю 3 курса 'Stepik Введение в Linux'

Дисциплина: Операционные системы

Мацюк Константин Владимирович

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение 3 модуля	7
3.1	Текстовый редактор vim	7
3.2	Скрипты на bash: основы	10
3.3	Скрипты на bash: ветвления и циклы	13
3.4	Скрипты на bash: разное	16
3.5	Продвинутый поиск и редактирование	19
3.6	Строим графики в gnuplot	23
3.7	Разное	25
4	Выводы	29
	Список литературы	30

Список иллюстраций

3.1	Выход из редактора vim	7
3.2	Различие между word и WORD	8
3.3	Преобразование строки в vim	9
3.4	Глобальная замена в vim	9
3.5	Команды в режиме Visual	10
3.6	История команд во вложенной оболочке	11
3.7	Определение абсолютного пути к файлу	11
3.8	Скрипт на bash, часть 1	12
3.9	Скрипт на bash, часть 2	12
3.10	Всегда истинные условия	13
3.11	Анализ условного оператора	14
3.12	Скрипт анализа количества студентов	14
3.13	Цикл for с командой continue	15
3.14	Бесконечный цикл с чтением данных	16
3.15	Использование команды let	16
3.16	Вывод текущего каталога через обратные кавычки	17
3.17	Проверка кода возврата программы	17
3.18	Функция counter и области видимости переменных	18
3.19	Реализация алгоритма Евклида	18
3.20	Калькулятор с обработкой ошибок	19
3.21	Поиск файлов с игнорированием регистра	20
3.22	Сравнение опций -path и -name	20
3.23	Сравнение вывода grep с разными опциями	21
3.24	Использование регулярных выражений	21
3.25	Дублирование строк в sed	22
3.26	Замена по шаблону в sed	22
3.27	Использование опции persist в gnuplot	23
3.28	Автоматическое определение заголовка в gnuplot	23
3.29	Создание пользовательских делений на оси X	24
3.30	Модификация анимации графика	25
3.31	Права доступа и числовое значение 764	26
3.32	Настройка прав для создания файлов в директории	26
3.33	Возможности команды wc	27
3.34	Вывод размера директории командой du	28
3.35	Генерация директорий с помощью фигурных скобок	28

Список таблиц

1 Цель работы

Пройти курс «Введение в Linux»

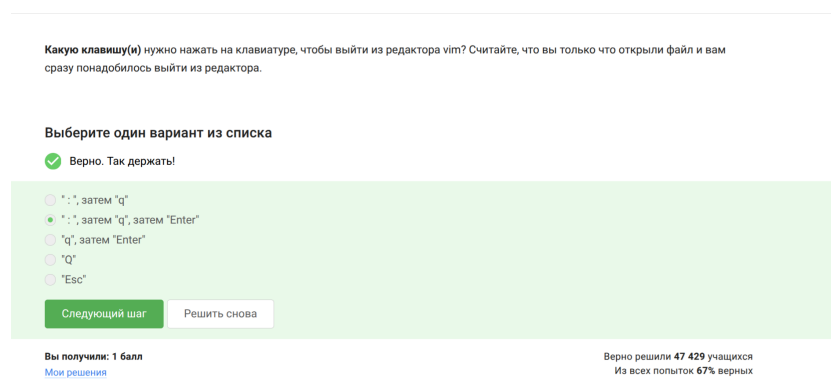
2 Задание

Выполнить модуль 3

3 Выполнение 3 модуля

3.1 Текстовый редактор vim

Для выхода из редактора vim сразу после открытия файла нужно нажать клавишу: , затем q и Enter, что переводит vim в командный режим и выполняет выход (рис. 3.1).



Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Верно. Так держать!

- ☐ ";", затем "q"
- ☒ ";", затем "q", затем "Enter"
- ☐ "q", затем "Enter"
- ☐ "Q"
- ☐ "Esc"

[Следующий шаг](#) [Решить снова](#)

Вы получили: 1 балл
[Мои решения](#)

Верно решили 47 429 учащихся
Из всех попыток 67% верных

Рис. 3.1: Выход из редактора vim

Различие между word и WORD: word состоит из букв, цифр и подчёркиваний, а WORD — из любых непробельных символов, поэтому нажатие w не может остановиться на точке, а для достижения конца строки требуется меньше нажатий W, чем w (рис. 3.2).

При перемещении в vim "по словам" есть небольшая разница в том, используем мы маленькую (w, e, b) или большую (W, E, B) букву. Первые перемещают нас по "словам" (word), а вторые по "большим словам" (WORD). Посмотрите справку по этим перемещениям и разберитесь в чем заключается разница между word и WORD.

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:

```
Strange_ TEXT is_here. 2=2 YES!
```

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать vim-справку по, например, перемещению `w`, нужно открыть vim и ввести команду `:help w`. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про `e` и про `b` и, самое главное, про word и WORD. Кроме того, можно вызвать сразу справку по термину word при помощи `:help word`. Чтобы закрыть справку, нужно ввести команду `:q`.

Выберите все подходящие ответы из списка

☒ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☒ В этой строке 9 "слов" (word)
 ☐ Нажимая только на w, нельзя переместить курсор на "."
 ☐ Чтобы попасть в конец строки, нужно одинаковое число нажатий, что на W, что на w
 ☐ В этой строке 5 "слов" (word)
 ☐ В этой строке 12 "слов" (word)
 ☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w

Следующий шаг
 Решить снова

Вы получили: 1 балл

Верно решили 36 720 учащихся

Рис. 3.2: Различие между word и WORD

Для преобразования строки использованы команды: `x2wwyuwPr` — удаление слова «one», перемещение и вставка; `d2wwyuwPr` — аналогично с удалением двух слов; `ddithree four four four five` — удаление строки и вставка новой; `d2wwifour four` — удаление до «three» и вставка «four four» (рис. 3.3).

Предположим, что в текстовом файле записана одна единственная строка:

```
one two three four five
```

и вам нужно преобразовать её в строку

```
three four four four five
```

Какие(ой) из предложенных ниже наборов нажатий клавиш выполнят такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка

☒ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в комментариях, отвечая на их вопросы, или сравнить своё решение с другими на форуме решений.

- ☒ ddthree four four four five<Esc>
- ☒ xxxxxxxxyuwPp
- ☒ d2wwfour four <Esc>
- ☐ d2dwywPp
- ☐ d2wwywrp
- ☒ d2wwywwPp

Следующий шаг

Решить снова

Вы получили: 1 балл
Мои решения

Верно решили 34 245 учащихся
Из всех попыток 16% верных

5 учащимся понравился этот шаг, а вам?
😊 😊 😊 😊 😊

Следующий шаг >

Рис. 3.3: Преобразование строки в vim

Команда `:%s/Windows/Linux/` выполняет глобальную замену первого вхожде-
ния (рис. 3.4).

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов получено

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод `:` в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст

☒ Хорошие новости, верно!

:%s/Windows/Linux/

Следующий шагРешить снова

Рис. 3.4: Глобальная замена в vim

В режиме Visual (открывается из нормального режима клавишей v) доступны команды перемещения, а также команды d (удалить) и у (скопировать) (рис. 3.5).

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов получено

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

✓ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ В режиме выделения можно использовать команды d (удалить) и у (скопировать)
- ☒ Режим выделения открывается из нормального режима по нажатию "v"
- ☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)
- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☐ Режим выделения открывается из любого другого режима по нажатию "v"
- ☐ Режим выделения открывается при помощи команды :visual

Следующий шаг Решить снова

Рис. 3.5: Команды в режиме Visual

3.2 Скрипты на bash: основы

Каждая оболочка bash имеет свою собственную историю команд, поэтому во вложенной оболочке будут доступны только команды, введенные непосредственно в ней (набор C) (рис. 3.6).

3.2 Скрипты на `bash`: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Вы прошли больше 80% курса, оставьте отзыв

Оставить отзыв Нет, спасибо

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попробуете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

✓ Прекрасный ответ.

- ☒ Только из набора C
- ☐ Только из набора B
- ☐ Только из набора A
- ☐ Никакие команды появляться не будут
- ☐ Из наборов B и C

Следующий шаг

Решить снова

Вы получили: 1 балл

Мои решения

Верно решили 44 285 учащихся

Из всех попыток 63% верных

Рис. 3.6: История команд во вложенной оболочке

Скрипт сначала переходит в `/home/bi/`, затем создаёт файл `file1.txt` в этой директории, поэтому абсолютный путь к созданному файлу – `/home/bi/file1.txt` (рис. 3.7).

3.2 Скрипты на `bash`: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Вы прошли больше 80% курса, оставьте отзыв

Оставить отзыв Нет, спасибо

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

✓ Правильно, молодец!

- ☒ `/home/bi/file1.txt`
- ☐ `/home/bi/Desktop/file1.txt`
- ☐ Никак (файла `file1.txt` не будет существовать после завершения работы скрипта)
- ☐ `/home/bi/Documents/file1.txt`

Следующий шаг

Решить снова

Вы получили: 1 балл

Мои решения

Верно решили 44 285 учащихся

Из всех попыток 63% верных

Рис. 3.7: Определение абсолютного пути к файлу

3.2 Скрипты на bash: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Вы прошли больше 80% курса, оставьте отзыв [Оставить отзыв](#) Нет, спасибо

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

☒ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

☒ variable_123
☒ _variable
☒ variable123
☒ variable
☒ VARIable
☐ var-i-able
☐ var@iabile

[Следующий шаг](#) [Решить снова](#)

Вы получили: 1 балл [Мое решение](#) Верно решили 39 720 учащихся Из всех попыток 25% верных

Рис. 3.8: Скрипт на bash, часть 1

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [arguments.sh](#).

Напишите скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1первый_аргумент $2второй_аргумент
```

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране будет:

```
Arguments are: $1one $2two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1three $2four
```

Подсказка: в случае проблем с решением задачи, обратите внимание на [подсказки](#).

Напишите программу. Тестируется через stdin → stdout

☒ Здорово, всё верно.

Теперь вам доступен [форум решений](#), где вы можете сравнить свое решение с другими.

```

1
2
3 #!/bin/bash
4 echo "Arguments are: '$1=$1 $2=$2'"
5
6
7

```

[Следующий шаг](#) [Решить снова](#)

Вы получили: 3 балла [Мое решение](#) Верно решили 36 929 учащихся Из всех попыток 40% верных

5 учащихся появились этот шаг, а вам?

[Следующий шаг](#) >

Рис. 3.9: Скрипт на bash, часть 2

Скрипт выводит строку с подстановкой первого и второго аргументов, при этом знак доллара перед `$1` экранируется, чтобы напечатать литерал `$1`: `echo «Arguments are: $1=$1 $2=$2»`. Например, при запуске `./script.sh one two` на экране появится `Arguments are: $1=one $2=two`

3.3 Скрипты на `bash`: ветвления и циклы

Условия `5 -ge 5` (5 больше или равно 5) и `-z ""` (пустая строка) всегда истинны независимо от параметров скрипта и значений переменных (рис. 3.10).

3.3 Скрипты на `bash`: ветвления и циклы 7 из 9 шагов пройдено 5 из 10 баллов получено

Вы прошли больше 80% курса, оставьте отзыв

Оставить отзыв Нет, спасибо

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [branching1.sh](#).

Предположим, вы пишете скрипт на `bash` и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
  echo "True"
fi
```

Вы можете вписать вместо `"..."` (внутри `[[ ... ]]` и не забудьте про пробелы после `[[` и перед `]]`) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

☒ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☐ `$# -gt 0`

☐ `$var1 == $var2 && $var1 != $var2`

☒ `-s $0`

☒ `5 -ge 5`

☐ `-z ""`

☐ `"n $1`

Следующий шаг

Решить снова

Рис. 3.10: Всегда истинные условия

При `var=3` ни одно из условий не выполняется (3 не больше 5, не меньше 3, не равно 4), поэтому выводится «four». При `var=5` условие `-gt` проверяет строгое неравенство (`5 > 5` ложно), затем `-lt 3` ложно, `-eq 4` ложно, снова срабатывает `else` и выводится «four». Таким образом, оба раза будет напечатано «four» (рис. 3.11).

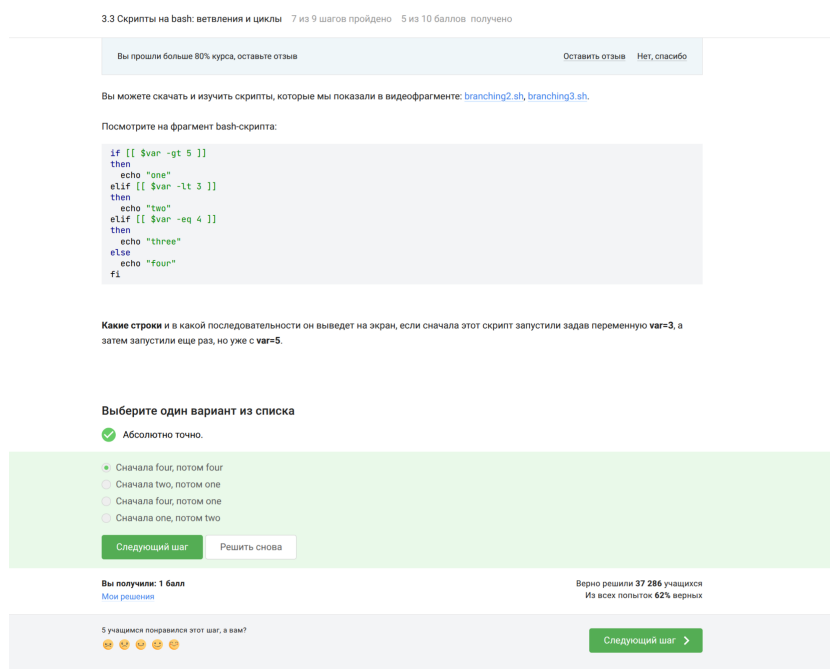


Рис. 3.11: Анализ условного оператора

Скрипт анализирует первый аргумент \$1: если 0 – «No students», если 1 – «1 student», если от 2 до 4 – «\$1 students», иначе «A lot of students». В коде важно использовать условие `-lt 5` для диапазона 2-4, потому что `-lt` означает «меньше» (рис. 3.12).

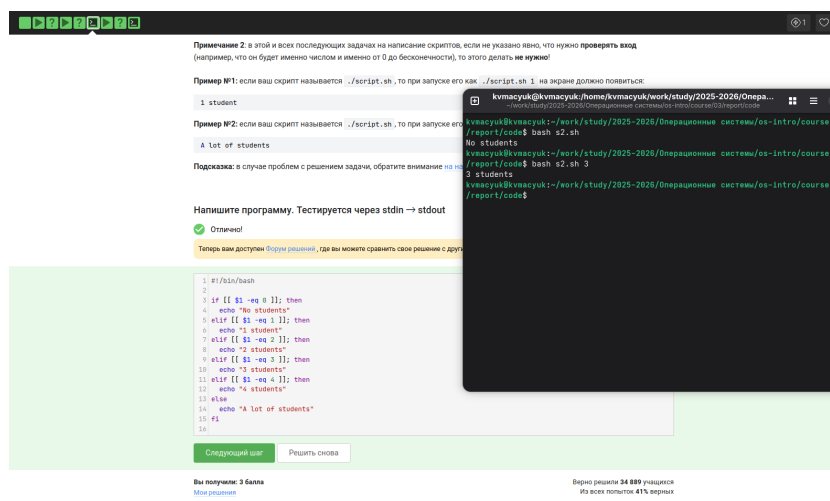


Рис. 3.12: Скрипт анализа количества студентов

Цикл `for` перебирает три элемента: «a», «b» и «c_d» (запятые входят в состав

строк). Команда `continue` пропускает вывод «finish» для тех строк, которые лексикографически больше «с». Строка «с_d» больше «с», поэтому «finish» не выводится. Остальные две строки выводят и «start», и «finish». Итого «start» появится 5 раз, а «finish» — 4 раза (рис. 3.13).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово "start", а сколько раз слово "finish"?

Выберите один вариант из списка

☒ Абсолютно точно.

☐ 5 раз "start" и ни разу "finish"

☐ 3 раза "start" и 2 раза "finish"

☐ 5 раз "start" и 2 раза "finish"

☒ 5 раз "start" и 4 раза "finish"

Рис. 3.13: Цикл `for` с командой `continue`

Скрипт в бесконечном цикле читает имя и возраст. Если имя пустое или возраст равен 0, выводится «bye» и цикл прерывается. Иначе проверяется возраст: если ≤ 16 — группа «child», если ≤ 25 — «youth», иначе «adult». После вывода сообщения цикл повторяется (рис. 3.14).

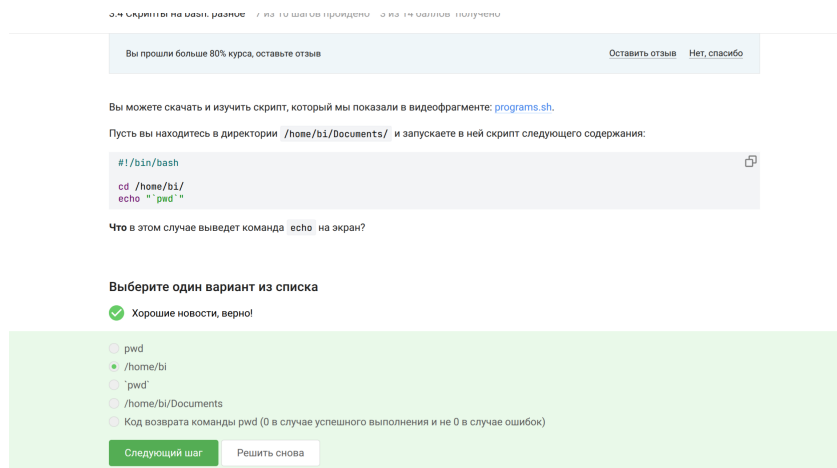


Рис. 3.16: Вывод текущего каталога через обратные кавычки

Чтобы проверить код возврата программы, её нужно запустить отдельно, а затем проверить переменную `?` (например, `if [[ $? -eq 0 ]]`). Нельзя использовать `if program`, потому что подстановка команды подставляет её вывод, а не код возврата (рис. 3.17).

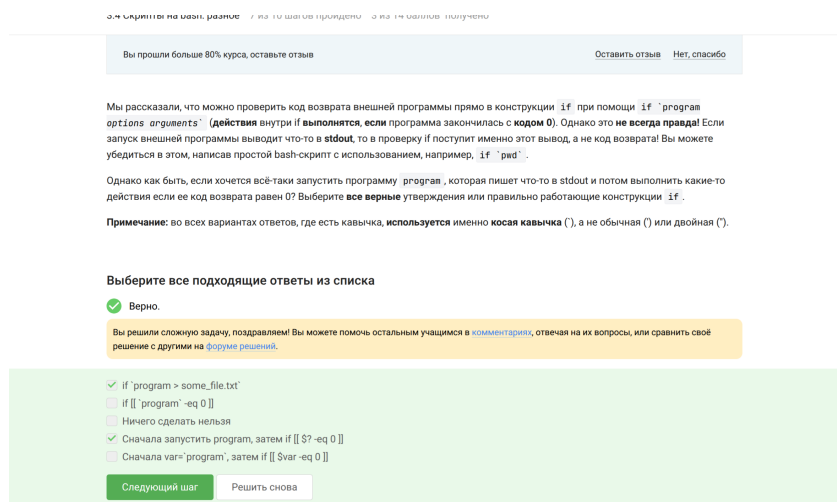


Рис. 3.17: Проверка кода возврата программы

В функции `counter` переменная `c1` объявлена как `local`, поэтому её значение не сохраняется между вызовами — после десяти вызовов `c1` остаётся пустой. Переменная `c2` не локальная, она накапливает сумму: $2 \cdot (1 + 2 + \dots + 10) = 110$. Команда `echo` выводит «counters are and 110» (между `are` и `and` два пробела) (рис. 3.18).

3.4 Скрипты на bash: разное 10 из 10 шагов пройдено 14 из 14 баллов получено

Вы прошли больше 80% курса, оставьте отзыв [Оставить отзыв](#) [Нет, спасибо](#)

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из bash-скрипта:

```
counter () # takes one argument
{
    local let "c1+=${1}"
    let "c2+=${1}*2"
}
```

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 😊). В этом случае имеет смысл написать небольшой скрипт на bash, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст

✓ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Рис. 3.18: Функция counter и области видимости переменных

Функция `gcd` реализует алгоритм Евклида через вычитание в цикле `while`. Пока `M` не равно `N`, из большего вычитается меньшее. После цикла выводится «GCD is ». Основная программа читает пары чисел через `read -r numbers`, затем с помощью `set - $numbers` разбивает строку на аргументы и вызывает `gcd`. При пустой строке выводится «bye» и выполнение прекращается (рис. 3.19).

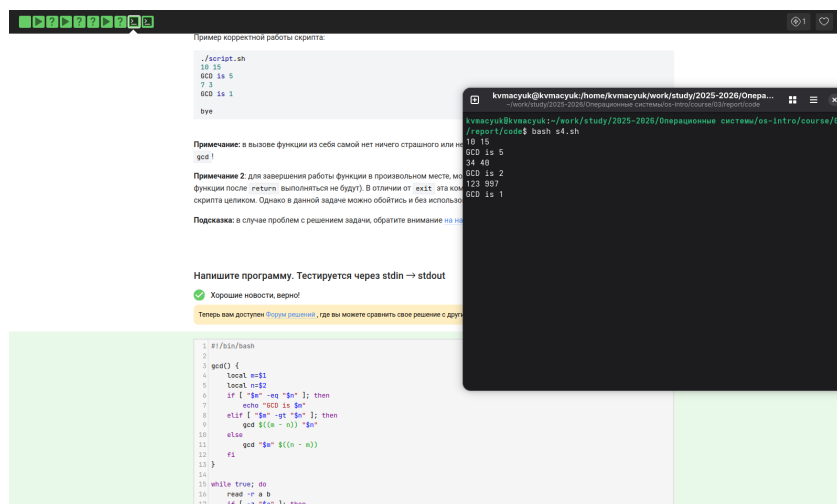


Рис. 3.19: Реализация алгоритма Евклида

Программа читает строки в цикле. Если введено «exit», выводится «bye» и цикл

прерывается. Иначе пытается вычислить выражение через `let r=((exp))`. Если вычисление успешно, выводится результат, иначе выводится «error» и скрипт завершается (рис. 3.20).

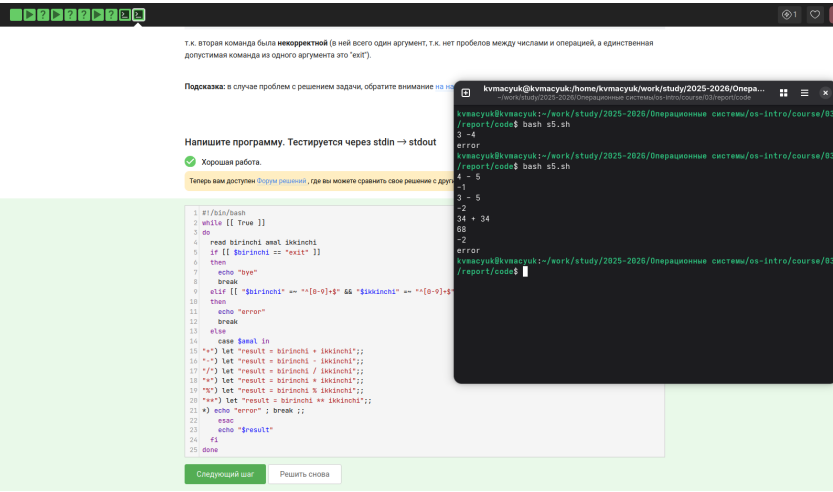


Рис. 3.20: Калькулятор с обработкой ошибок

3.5 Продвинутый поиск и редактирование

Опция `-iname` игнорирует регистр, поэтому найдутся все файлы, начинающи-
еся с «star» в любом регистре. Опция `-name` учитывает регистр. Команда `find`
`-iname «star*»` найдёт все четыре файла, начинающихся на Star/STAR/star. Раз-
ница — все четыре. В списке ответов отмечены STARS.txt и Star_Wars.avi (рис.
3.21).

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Вы прошли больше 80% курса, оставьте отзыв [Оставить отзыв](#) [Нет, спасибо](#)

Пусть в директории `/home/b1` лежат файлы `Star_Wars.avi`, `star_trek OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/b1 -iname "star*"`, но **НЕ найдет** команда `find /home/b1 -name "star*"` ?

Выберите все подходящие ответы из списка

☒ Правильно.

- ☐ `star_trek OST.mp3`
- ☒ `STARS.txt`
- ☐ `stardust.mpeg`
- ☐ `Eddard_Stark_biography.txt`
- ☒ `Star_Wars.avi`

[Следующий шаг](#) [Решить снова](#)

Рис. 3.21: Поиск файлов с игнорированием регистра

Опции `-path` и `-name` отличаются: `-path` сопоставляет весь путь, а `-name` только имя файла. Их замена иногда даёт одинаковый результат, например, когда файл находится в корне поиска. Утверждение «Если заменить `-name` на `-path`, результат поиска иногда может остаться таким же» верно (рис. 3.22).

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Вы прошли больше 80% курса, оставьте отзыв [Оставить отзыв](#) [Нет, спасибо](#)

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

- ☐ Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- ☐ Если заменить в команде поиска `-name` на `-path`, то результат поиска всегда останется неизменным
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☒ Если заменить в команде поиска `-name` на `-path`, то результат поиска иногда может остаться таким же
- ☐ Опции `-path` и `-name` всегда работают одинаково

[Следующий шаг](#) [Решить снова](#)

Рис. 3.22: Сравнение опций `-path` и `-name`

Во всех вариантах `grep` выводит одинаковое количество строк, потому что каждая строка файла содержит слово «word». Опции `-A`, `-B`, `-C` добавляют контекст, но так как в каждой строке есть искомое слово, то добавленные строки контекста — это те же самые строки, которые уже выводятся. В итоге все команды создадут файл `results.txt` одинакового размера (рис. 3.23).

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Вы прошли больше 80% курса, оставьте отзыв [Оставить отзыв](#) [Нет, спасибо](#)

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```

/home/bi/
├── dir1
│   ├── file1
│   └── dir2
│       ├── file2
│       └── dir3
│           └── file3

```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Выберите один вариант из списка

☒ Так точно!

- ☐ Только `file3`
- ☐ Все кроме `file1`
- ☐ Все кроме `file2`
- ☐ Только `file1`
- ☒ Все кроме `file3`

[Следующий шаг](#) [Решить снова](#)

Рис. 3.23: Сравнение вывода `grep` с разными опциями

Регулярное выражение ищет строки, заканчивающиеся определённым образом (рис. 3.24).

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Вы прошли больше 80% курса, оставьте отзыв [Оставить отзыв](#) [Нет, спасибо](#)

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем **в каждой строке есть слово "word"**. Если вы выполните на этом файле команды:

```

grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt

```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

☒ Верно.

- ☐ `grep -C 1 "word" file.txt > results.txt`
- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях
- ☐ `grep -A 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`

[Следующий шаг](#) [Решить снова](#)

Вы получили: 1 балл [Мои решения](#)

Верно решили 30 487 учащихся
Из всех попыток 42% верных

5 учащимся понравился этот шаг, а вам? [Следующий шаг](#)

Рис. 3.24: Использование регулярных выражений

Без опции `-n` `sed` печатает каждую строку по умолчанию, а команда `r` печатает строку ещё раз. Строки, соответствующие шаблону `[a-z]*` (любая строка, содержащая хотя бы одну строчную букву), будут выведены дважды. Если строка не

соответствует шаблону, она будет напечатана один раз (рис. 3.25).

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Вы прошли больше 80% курса, оставьте отзыв [Оставить отзыв](#) [Нет, спасибо](#)

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xkLXkL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

✓ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Mac OS X 10.9, Windows XP, Ubuntu 12.04
- ☒ I prefer Kubuntu
- ☐ Kubuntu
- ☒ The best OS is Xubuntu
- ☒ Linux is not always Ubuntu
- ☒ Hmm, XKcLubuntu

[Следующий шаг](#) [Решить снова](#)

Вы получили: 2 балла [Мои решения](#)

Верно решили 28 093 учащихся
Из всех попыток 23% верных

3 учащимся понравился этот шаг, а вам? 😊 😊 😊 😊 😊

[Следующий шаг](#) >

Рис. 3.25: Дублирование строк в sed

Инструкция sed `„s/[A-Z][A-Z]+ / abbreviation /g“ input.txt > edited.txt` заменяет последовательности из двух и более заглавных букв, окружённые пробелами, на слово `abbreviation`. Пробелы до и после сохраняются (рис. 3.26).

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Вы прошли больше 80% курса, оставьте отзыв [Оставить отзыв](#) [Нет, спасибо](#)

Что произойдет, если в команде `sed -n "/[a-z]*p" text.txt` не указывать опцию `-n`?

Выберите один вариант из списка

✓ Хорошие новости, верно!

- ☒ Каждая строка будет выведена два раза
- ☐ На экран ничего не напечатается
- ☐ Будут выведены все строки файла `text.txt`, в которых есть только большие буквы латинского алфавита
- ☐ Появится сообщение об ошибке

[Следующий шаг](#) [Решить снова](#)

Рис. 3.26: Замена по шаблону в sed

3.6 Строим графики в gnuplot

Опция -p (–persist) при запуске gnuplot предотвращает закрытие окон с графиками после завершения работы программы (рис. 3.27).

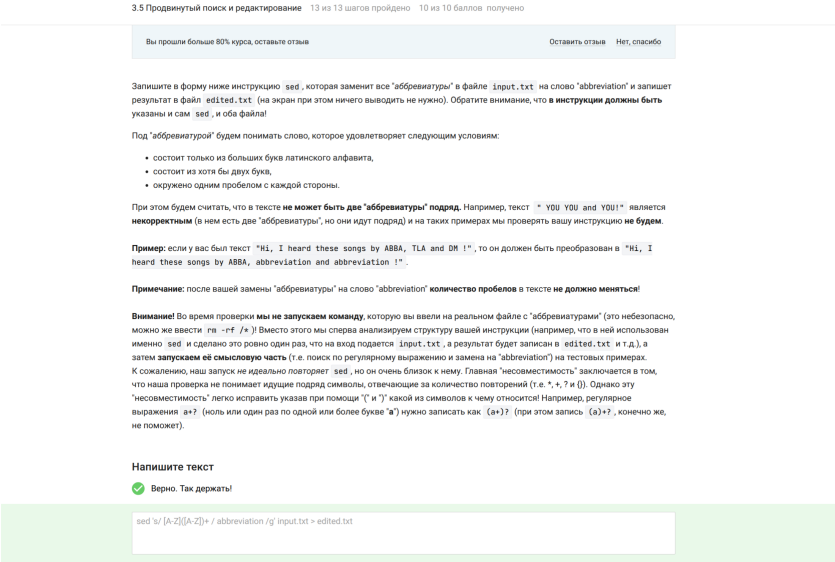


Рис. 3.27: Использование опции persist в gnuplot

При использовании set key autotitle columnhead и отсутствии заголовков в первой строке, gnuplot воспринимает первую строку как данные, а не как заголовок. Поэтому название ряда берётся из первого значения второго столбца (первая строка, второе поле). Всего точек будет нарисовано 9, потому что первая строка использована для названия и не отображается как точка (рис. 3.28).

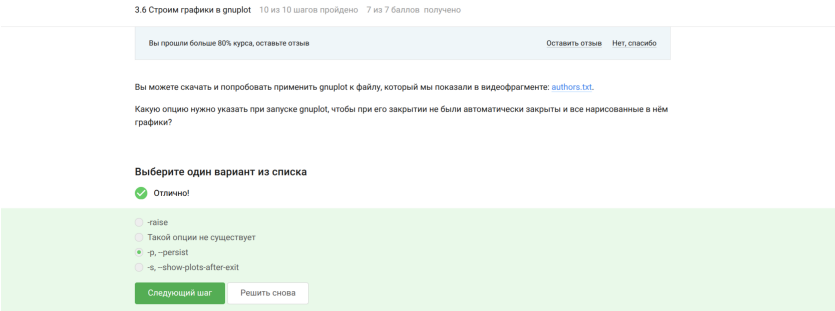


Рис. 3.28: Автоматическое определение заголовка в gnuplot

Команда set xtics создаёт пользовательские деления на оси X. Конкатенация

строк: `set xtics ('point 1, value'.x1 x1, 'point 2, value'.x2 x2, 'point 3, value'.x3 x3)` (рис. 3.29).

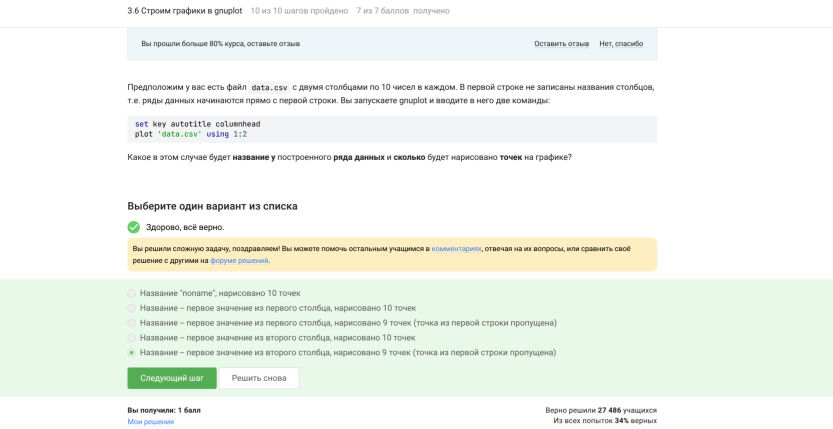


Рис. 3.29: Создание пользовательских делений на оси X

Для отражения графика относительно горизонтальной плоскости функция изменена на `-x2-y2` (вместо `x2+y2`). Обратное вращение достигается изменением приращения угла: `zrot=(zrot+350)%360` (350 эквивалентно -10). Ускорение в два раза — уменьшением паузы с 0.2 до 0.1 секунды (рис. 3.30).

Вы прошли больше 80% курса, оставьте отзыв

Оставить отзыв Нет, спасибо

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [plot.gnu](#), [plot_advanced.gnu](#), [plot_advanced2.gnu](#). Все три скрипта основаны на [этой заметке](#), данные также взяты оттуда.

Предположим, что вы пишете gnuplot-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде **"point <номер точки>, value <значение соответствующей переменной>"**. Например, для `x1=0`, `x2=10`, `x3=20`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20. Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно**!

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин **конкатенация**, который важен для выполнения данного задания. Под конкатенацией обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишите текст

✓ Всё правильно.

```
set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)
```

Следующий шаг

Решить снова

Рис. 3.30: Модификация анимации графика

3.7 Разное

Права `rwxr-xr` соответствуют числовому значению 764. Правильные команды: `chmod a+wx file.txt`; `chmod o-wx file.txt`; `chmod g-x file.txt`; `chmod ug+w file.txt`; `chmod u+x file.txt`; `chmod 764 file.txt`; `chmod u+wx file.txt`; `chmod g+w file.txt` (рис. 3.31).

3.6 Строим графики в gnuplot 10 из 10 шагов пройдено 7 из 7 баллов получено

Если вы не скачали на предыдущем шаге файлы [animated.gnu](#) и [move.rot](#), то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять** и **удалять** инструкции **нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу `gnuplot` и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в `gnuplot` работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Напишите текст

✔ Отличное решение!

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Следующий шаг Решить снова

Вы получили: 3 балла
Мои решения

Верно решили 21 244 учащихся
Из всех попыток 55% верных

Рис. 3.31: Права доступа и числовое значение 764

Чтобы пользователь `user` из группы `group` мог создавать файлы в директории, нужно либо сменить владельца (`sudo chown user dir` или `sudo chown user:group dir`), либо добавить право записи для группы или всех с `sudo`: `sudo chmod o+w dir` или `sudo chmod a+w dir`. Просто `chmod o+w` без `sudo` не работает, так как директория принадлежит `root` (рис. 3.32).

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Какая команда(ы) установят файлу `file.txt` права доступа `rwXrw-r--`, если изначально у него были права `r--r--r--`. Укажите **все верные** варианты ответа!

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнились все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Выберите все подходящие ответы из списка

✔ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☒ `chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt`
☒ `chmod ug+w file.txt; chmod u+x file.txt`
☐ `chmod rwxrw-r-- file.txt`
☐ `chmod 467 file.txt`
☒ `chmod 764 file.txt`
☒ `chmod u+wx file.txt; chmod g+w file.txt`

Следующий шаг Решить снова

Рис. 3.32: Настройка прав для создания файлов в директории

Команда `wc` (word count) может подсчитать количество байт (опция `-c`), количество слов (`-w`) и количество символов (`-m`). Она не умеет считать предложения или отдельные буквы (рис. 3.33).

Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `rw-r--r--` (владелец `root`, группа `root`). Таким образом никто кроме пользователя `root` не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды `user` из группы `group` всё-таки сможет создать файл внутри `dir`? Укажите **все верные** варианты ответов!

Примечание: считаем, что все команды выполняются от имени `user`, если явно не указано, что команда выполнена с `sudo`.

Примечание 2: мы выбрали пример с директорией, а не с файлом не случайно. Дело в том, что если создать при помощи `sudo` файл с правами `rw-r--r--` в директории, которая принадлежит пользователю, то возникнет любопытная ситуация. С одной стороны пользователь может удалить этот файл (т.к. ему разрешено удалять **все** файлы внутри его директории) и может прочитать его содержимое (т.к. право `r` у файла установлено для всех), с другой стороны он не может этот файл редактировать (т.к. право `w` у файла есть только для `root`). При этом некоторые "умные" редакторы, например, `vim` позволят даже редактировать этот файл, но сделают они это своеобразно: через удаление оригинала и создание копии уже с нужными правами (удалять мы можем, а раз можем читать, то и копию создать не сложно). Итого получается, что несмотря на права `rw-r--r--`, пользователь может сделать с этим файлом почти всё что угодно! В случае же, когда речь идет о директории созданной `root`, ситуация будет проще: пользователь сможет смотреть её содержимое (у него есть право `r`), но удалять и создавать файлы в ней не сможет (права `w` у него нет). Важно отметить, что директории в `Linux` это в каком-то смысле файлы. Содержимое такого "файла" -- это записи о файлах и поддиректориях этой директории (грубо говоря их названия). Таким образом, право `r` у директории дает возможность просматривать "записи", т.е. просматривать её состав. Право `w` у директории дает возможность удалять/добавлять новые "записи", т.е. удалять/создавать файлы/поддиректории в ней. На самом деле и это еще не всё. Существует так называемый `sticky bit` (атрибут файла или директории), выставление которого меняет описанное выше поведение. Файлы (или директории) с таким атрибутом сможет удалить только их владелец вне зависимости от прав, установленных у директории, в которой эти файлы (или директории) лежат!

Отдельное спасибо слушателю курса **Alexey Antipovsky** за помощь в оформлении **Примечания 2!**

Выберите все подходящие ответы из списка

☒ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☒ `sudo chown user dir`

☐ `chmod o+w dir`

☒ `sudo chmod o+w dir`

☒ `sudo chmod a+w dir`

☒ `sudo chown user:group dir`

☐ `sudo chown .group dir`

Следующий шаг

Решить снова

Рис. 3.33: Возможности команды `wc`

Команда `du -sh` (или `du -h -s`) выводит суммарный размер текущей директории (рис. 3.34).

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

✓ Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Размер файла в байтах
- ☒ Количество слов
- ☐ Количество предложений
- ☒ Количество символов
- ☐ Количество определенных букв (например, количество букв "А")

Следующий шаг Решить снова

Рис. 3.34: Вывод размера директории командой `du`

Самая короткая команда для создания трёх поддиректорий — `mkdir dir{1..3}`. Фигурные скобки в `bash` генерируют последовательность: `dir1`, `dir2`, `dir3` (рис. 3.35).

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести **в удобном для чтения формате** (например, вместо `2048 байт` надо выводить `2.0K`) и **больше** на экран выводить **ничего не** нужно). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по `800 Кбайт` и две поддиректории в каждой из которой лежит по файлу в `400 Кбайт`, то загаданная команда должна вывести на экран одно число: `2.4M` (также на экране может быть выведен еще и символ `..`, обозначающий, что это размер именно текущей директории).

Напишите текст

✓ Правильно, молодец!

`du -h -s`

Следующий шаг Решить снова

Рис. 3.35: Генерация директорий с помощью фигурных скобок

4 Выводы

Я прошел модуль 3 курса Stepik

Список литературы